

THE ENTERPRISE PR REVIEW PLAYBOOK

Volume II — Deep Domain Reviews

Architecture, ADRs and RFCs, Security, Infrastructure, Databases,
APIs, and Documentation — Reviewed the Way Principal Engineers Actually
Read Them

A practitioner-level reference on domain-specific review discipline, grounded in published standards (OWASP Top 10:2025, SLSA, ADR/RFC practice) and engineering literature from Google, AWS, Microsoft, Stripe, Cloudflare, and the open-source security community.

Series: 5 Volumes · Volume 2 of 5
July 2026

Contents

Section 3 — Architecture Review

- 3.1 What architecture review actually looks for in a diff
- 3.2 Service and API boundary changes
- 3.3 Event contracts and schema evolution
- 3.4 Database migration boundary concerns

Section 4 — Security Review

- 4.1 The OWASP Top 10:2025 lens
- 4.2 AuthN, AuthZ, and session management
- 4.3 AI-era attack surfaces: prompt and agent injection, MCP, A2A
- 4.4 Supply chain: SBOM, SLSA, Sigstore, signed commits
- 4.5 Manual vs. automated security checklist

Section 5 — Infrastructure Review

- 5.1 Terraform, Pulumi, CloudFormation review
- 5.2 Kubernetes, Helm, and Dockerfiles
- 5.3 CI/CD pipeline review (GitHub Actions, GitLab CI, ArgoCD, Flux)
- 5.4 Zero Trust, RBAC, cost, and autoscaling

Section 6 — Database Review

Section 7 — API Review (REST, GraphQL, gRPC, Async)

Section 8 — Documentation Review

Section 9 — ADR-Driven Development

Section 10 — RFC-Driven Engineering

About This Series

Section 3 — Architecture Review

Architecture review inside a PR is not a separate ceremony from code review — it is a different lens applied to the same diff. A reviewer doing architecture review is asking whether this change is consistent with decisions the organization has already made, and whether it's creating a new decision that hasn't been recorded anywhere.

3.1 What Architecture Review Actually Looks For

- **ADR/RFC references** — does the PR description link to the design doc or decision record that justifies this approach? A PR introducing a new dependency direction, a new data store, or a new cross-service call pattern without a linked ADR is a signal to slow down, not a formality to skip.
- **Sequence and event-flow correctness** — for anything asynchronous, does the diff match the sequence diagram in the design doc, or has the implementation quietly diverged from what was agreed?
- **Dependency direction** — does this change introduce a dependency that violates the intended layering (e.g., a domain-layer module now importing from an infrastructure-layer module)?
- **Bounded-context integrity** — in a DDD-organized system, does this change respect the boundary of the domain that owns the data, or does it reach across a boundary to read or write another domain's internal model directly instead of going through its published interface?
- **Microservice granularity** — is this PR growing an existing service in a way that blurs its single responsibility, effectively merging two services' concerns into one deployable unit?

3.2 Service and API Boundary Changes

The riskiest class of architectural change is one that alters a boundary other teams depend on without those teams being aware it happened. Reviewers at this layer check: is this API change additive or breaking; is there a deprecation window; has the change been announced to known consumers; and — critically — does the reviewer actually know who the consumers are, or is that knowledge missing entirely (a common and dangerous gap in fast-growing polyrepo organizations without a service catalog).

3.3 Event Contracts and Schema Evolution

For event-driven systems (Kafka, Kinesis, Pub/Sub, or similar), the review question is whether a schema change is compatible with every consumer still running the old schema — not just the producer's own tests. Consumer-driven contract testing (e.g., Pact-style contracts) is the mechanism mature organizations use to make this checkable in CI rather than dependent on a reviewer's memory of who consumes a topic.

- Adding an optional field to an event schema is generally safe; removing or renaming a field, or changing a field's type or semantic meaning, is generally breaking and needs a versioned schema or a dual-publish transition period.
- Schema registries (Confluent Schema Registry, AWS Glue Schema Registry) that enforce compatibility modes (backward, forward, full) at the CI/publish level turn this from a review-time judgment call into an automated gate — the more mature pattern.

3.4 Database Migration Boundary Concerns

Architecture review and database review overlap heavily here: a schema migration that looks correct in isolation can still be an architectural regression if it couples two services to the same physical schema, re-introducing the tight coupling that a service boundary was meant to remove in the first place. See Section 6 for migration mechanics.

Section 4 — Security Review

Security review operates on two tracks simultaneously: automated scanning that runs on every PR regardless of content, and targeted human review triggered by risk signals (auth changes, new external inputs, new dependencies, new data stores touching regulated data). The OWASP Top 10:2025 release reorganized the landscape in ways that materially change what a security reviewer should be looking for versus the prior 2021 edition.

4.1 The OWASP Top 10:2025 Lens

OWASP Top 10:2025 — What Changed and Why It Matters to Reviewers

Category	Reviewer Implication
A01 — Broken Access Control	Still #1. Now absorbs Server-Side Request Forgery (SSRF), which was previously its own category — reflecting that SSRF is fundamentally an access-control failure (the server accessing a resource it shouldn't on the attacker's behalf).
A02 — Security Misconfiguration	Jumped from #5 to #2, now affecting a measurably larger share of tested applications — a direct consequence of the shift to cloud-native, config-heavy infrastructure where a single wrong IAM policy or open bucket is catastrophic.
A03 — Software Supply Chain Failures	New top-3 category, expanding the old "vulnerable components" category to the full pipeline: compromised dependencies, tampered build steps, stolen signing keys, and over-permissive CI/CD integrations. Ranked #1 by over half of practitioners surveyed for the standard.
A04 — Cryptographic Failures	Dropped from #2 to #4 as baseline TLS/crypto hygiene has improved industry-wide, but remains a hard blocker whenever a reviewer sees custom cryptography or a legacy cipher suite.
A05 — Injection	Fell from #3 to #5 — still critical, but no longer the dominant category as parameterized queries and modern ORMs have closed off much of the easy surface.
A10 — Mishandling of Exceptional Conditions (new)	A new category covering improper error handling, logic errors, and fail-open conditions — the class of bug where a system's behavior under failure is itself the vulnerability, not a component the system depends on.

Source: OWASP Top 10:2025 official release (owasp.org/Top10/2025). The 2025 edition also flags, as a forward-looking "next steps" item rather than a ranked category, the risk of unreviewed AI-generated code being merged without a human fully understanding its logic.

4.2 AuthN, AuthZ, and Session Management

- Every new endpoint: is authorization checked at the resource level, not just "is the user logged in"? The most common real-world finding is an endpoint that checks authentication but not whether *this* authenticated user is allowed to act on *this specific* resource — classic broken object-level authorization.

- Token handling: are JWTs validated for signature, issuer, audience, and expiry on every use, not just decoded and trusted?
- Session fixation and rotation: does a privilege change (e.g., login, password reset) rotate the session identifier?
- RBAC/ABAC model correctness: does a new role or permission actually map to the intended resource scope, or does it accidentally grant broader access than described in the PR?
- Multi-tenancy isolation: for shared-infrastructure SaaS systems, is tenant ID enforced at the query layer (not just the application layer), so a bug elsewhere in the code cannot leak cross-tenant data?

4.3 AI-Era Attack Surfaces

- **Prompt injection** — user-controlled or externally-fetched text that enters a system prompt or tool-calling context without clear delimiting; a reviewer should be able to point to exactly where trusted instructions end and untrusted content begins.
- **Agent injection** — the multi-step analog of prompt injection, where a malicious instruction embedded in a document, webpage, or tool result causes an agent to take an unintended action later in its reasoning chain, not just produce bad text.
- **MCP (Model Context Protocol) security** — does a newly integrated MCP server request more tool scope than the task needs; is the server's identity and provenance verified before granting it access to sensitive tools or data?
- **A2A (agent-to-agent) trust boundaries** — when one agent's output becomes another agent's input, is that boundary treated as untrusted external input (validated, scoped) or implicitly trusted because "it's our own system"? The latter is the mistake reviewers should catch.

4.4 Supply Chain: SBOM, SLSA, Sigstore, Signed Commits

Software supply chain review has become one of the highest-leverage areas of security review, driven by a string of high-profile 2025 incidents — a self-propagating npm worm that compromised hundreds of downstream packages by harvesting maintainer credentials, and a compromised GitHub Action that was pinned by tag rather than commit SHA, silently pulling malicious code into tens of thousands of repositories the moment its tag was overwritten.

Supply-Chain Controls a Reviewer Should Expect to See

Control	What It Provides
SBOM (Software Bill of Materials)	An inventory of every component in a build — the "ingredient list." Reviewers should expect a signed SBOM (CycloneDX or SPDX format) generated as part of every release build, not reconstructed after the fact.
SLSA (Supply-chain Levels for Software Artifacts)	A framework, originally developed inside Google and now under the OpenSSF, defining build-integrity levels. Level 2 (hosted build service, signed provenance) is the realistic near-term target for most production software; Level 3 (hermetic, isolated builds with non-falsifiable provenance) is the bar for regulated or high-risk components.

Control	What It Provides
Sigstore (cosign, Fulcio, Rekor)	The standard open-source toolchain for keyless artifact signing: Fulcio issues short-lived certificates tied to an OIDC identity, Cosign signs the artifact, and Rekor records the signing event in an immutable transparency log — removing the operational burden of managing long-lived signing keys.
Pinning by commit SHA, not tag	The single highest-leverage low-cost control after the 2025 compromised-GitHub-Action incidents: a tag can be silently repointed by a compromised maintainer account; a commit SHA cannot.

Sources: *SLSA specification (slsa.dev)*, *Sigstore project documentation*, *OWASP Top 10:2025 A03 category writeup*.

4.5 Manual vs. Automated Security Checklist

Manual Checklist (Human Judgment Required)

- Does this endpoint check resource-level authorization, not just authentication?
- Is every secret pulled from a secrets manager, never hardcoded or logged?
- Is user input that reaches a shell, SQL query, or template engine parameterized or escaped?
- Does a new external call have a defined trust boundary and input validation at that boundary?
- Is PII classified and handled according to its classification (encryption, retention, access logging)?
- For a new dependency: is it from a vetted source, pinned to a specific version, and does its provenance check out (not typosquatting an established package)?

Automated Checklist (Should Never Depend on a Human Remembering)

- SAST (static analysis) findings on the diff, surfaced inline in the review tool.
- Secret-scanning pre-commit and pre-merge, blocking on any match.
- Dependency / SCA scanning against known-vulnerability databases (NVD, OSV.dev).
- DAST or fuzz testing against staging environments for externally-facing endpoints.
- SBOM generation and signature verification as a required CI step, not optional tooling.
- License scanning to catch copyleft-incompatible dependencies before they ship.

Section 5 — Infrastructure Review

5.1 Terraform, Pulumi, CloudFormation Review

- **Plan output review, not just diff review** — the actual risk in an IaC PR is often invisible in the code diff and only visible in the `terraform plan` output (e.g., a change that looks like a rename in code but is actually a destroy-and-recreate at the provider level).`
- **State and blast radius** — does this change affect a shared module used by multiple environments, meaning a single merge could quietly change production and staging simultaneously?
- **Least-privilege IAM** — is a new IAM policy scoped to specific resources and actions, or does it use a wildcard (*) that will be flagged in every future security audit?
- **Secrets in state** — does this introduce a resource whose sensitive output (a generated password, a private key) will be stored in plaintext in Terraform state unless explicitly marked sensitive and the backend encrypted?

5.2 Kubernetes, Helm, and Dockerfiles

Kubernetes / Container Review Checklist

Item	Why It's Reviewed
Resource requests/limits	Missing or unrealistic requests/limits are the single most common cause of noisy-neighbor incidents — a pod without limits can starve every other workload on its node.
Liveness/readiness probes	Missing or misconfigured probes mean Kubernetes cannot tell a hung process from a healthy one, defeating the platform's own self-healing.
Non-root containers	Dockerfiles that run as root by default are a standing privilege-escalation risk if the container is ever compromised; reviewers should expect an explicit non-root USER.
Image provenance	Is the base image pinned to a digest (not <code>latest</code>), from a trusted registry, and ideally signed and verifiable at admission time via a policy controller?
Rollback safety	Does the deployment strategy (rolling update, blue/green, canary) allow a fast, automated rollback, or does a bad deploy require manual intervention to reverse?

5.3 CI/CD Pipeline Review

The pipeline definition itself (GitHub Actions workflows, GitLab CI configs, ArgoCD/Flux application manifests) is code, and increasingly the highest-value target for an attacker, since it often runs with deployment credentials. Review should treat pipeline changes with at least the scrutiny of production application code, and often more, since a compromised pipeline can affect every downstream deployment.

- Are third-party GitHub Actions pinned by commit SHA rather than a mutable tag or branch name?
- Do pipeline runners use ephemeral, isolated environments rather than long-lived shared runners that could retain secrets or artifacts between jobs?

- Are deployment credentials scoped per-environment and short-lived (OIDC-based, not long-lived static keys checked into a secrets store forever)?
- For GitOps tools (ArgoCD, Flux): does a change to the desired-state repository require the same review rigor as application code, since merging to that repo is functionally equivalent to deploying?

5.4 Zero Trust, RBAC, Cost, and Autoscaling

- Zero Trust posture: does this change assume network location implies trust (e.g., "it's fine, it's inside the VPC"), or does it enforce identity-based verification regardless of network position?
- RBAC scope creep: does a new cluster role or service account request broader permissions than the workload actually needs, "to be safe"?
- Cost review: does a new autoscaling configuration have a sane maximum, or can a traffic spike (or a bug causing a request loop) scale the infrastructure bill without bound?
- Autoscaling correctness: are scale-up and scale-down thresholds tuned to avoid flapping (rapid scale up/down cycles that increase cost and reduce stability without improving capacity)?

Section 6 — Database Review

Database changes are disproportionately dangerous in review because their failure mode is often irreversible or extremely expensive to reverse — unlike application code, which can usually just be rolled back. A database reviewer's default posture is paranoid, and for good reason.

Database Migration Review Checklist

Concern	Reviewer Question
Indexes	Does a new query pattern have a supporting index? Conversely, does a migration drop an index that something still depends on? Index creation on a large table should default to an online/concurrent build to avoid locking writes.
Locks and transactions	Does this migration take a long-held lock on a large, high-traffic table (e.g., adding a column with a default value on some database engines can rewrite the whole table under lock)? Is the transaction scope as narrow as possible?
Backward compatibility / roll-forward-only	Can the application run against both the old and new schema simultaneously during a rolling deploy? If not, this is a coordinated-deployment risk, not a routine merge.
Rollback strategy	Is there a tested down-migration, or is this a one-way door (e.g., a destructive column drop) that needs an explicit backup/snapshot step before it runs?
Large-table operations	Has the migration been tested against production-scale data volume, not just a small development database where the same operation completes instantly?
Partitioning and CDC	Does this change interact with existing partitioning strategy or change-data-capture (CDC) pipelines that downstream systems depend on for replication or analytics ingestion?
Replication lag	For a write-heavy migration, has the reviewer considered the impact on replication lag to read replicas that other services depend on for read consistency?

The **expand/contract pattern** is the standard mitigation for nearly every risk in this table: add the new column/table/index alongside the old one (expand), migrate reads and writes over in a separate, reversible step, then remove the old structure only once nothing references it (contract). A reviewer seeing a migration that tries to do all three steps in a single PR should generally ask for it to be split.

Section 7 — API Review (REST, GraphQL, gRPC, Async)

7.1 Cross-Cutting API Concerns

- **Contract-first discipline** — was the OpenAPI/protobuf/GraphQL schema change reviewed and agreed before implementation, or does the schema get generated from whatever the implementation happened to produce (a common source of accidental breaking changes)?
- **Versioning strategy** — is a breaking change versioned explicitly (new endpoint version, new field deprecation cycle) rather than mutating the existing contract in place?
- **Pagination** — does a new list endpoint paginate by default, or does it return an unbounded result set that will become a performance and cost problem as data grows?
- **Error model consistency** — does this endpoint's error response follow the same shape (status codes, error body schema) as the rest of the API, or does it invent a bespoke format that every client now has to special-case?
- **Idempotency** — for any mutating operation that a client might retry (payments being the canonical example), is there an idempotency key mechanism so a retried request doesn't double-execute?
- **Rate limiting** — is this endpoint covered by existing rate-limit policy, or does it bypass it because it's "internal only" (a classification that has a way of becoming untrue over time)?
- **Observability** — does this endpoint emit the standard request/latency/error metrics and distributed tracing spans by default, or will it be invisible in the dashboards until someone notices it's missing during an incident?

7.2 By Protocol

Protocol-Specific Review Focus

Protocol	What Reviewers Check
REST / OpenAPI	Reviewer checks the OpenAPI diff for removed fields, changed types, or tightened validation on existing fields — all of which are breaking even if the endpoint path is unchanged. Tooling (e.g., openapi-diff style checks) can gate this automatically in CI.
GraphQL	Field deprecation via <code>@deprecated</code> rather than removal; N+1 query risk from a new resolver that doesn't batch/dataloader; whether a new field exposes more data than the client actually needs, widening the effective API surface silently.
gRPC / Protobuf	Field number reuse is the classic breaking mistake — removing a field and reusing its number for something new corrupts data for any client still on the old schema. Reviewers check that removed fields are reserved, not reused.
Async / Event APIs	See Section 3.3 — compatibility mode enforcement via schema registry, and whether a new consumer needs to handle both old and new event shapes during a transition window.

Section 8 — Documentation Review

Documentation review is frequently treated as optional in PR review, which is precisely why it decays. Mature organizations treat a documentation gap the same way they'd treat a missing test: a reason to request changes, not a nice-to-have.

Documentation Review Checklist

Artifact	Reviewer Question
README	Does it still accurately describe how to run and test the system after this change, or does the PR silently make the README wrong?
ADRs	Does an architecturally significant change have a corresponding ADR, linked from the PR description? (See Section 9.)
Runbooks / playbooks	For a change that affects on-call response (new failure mode, new alert, changed rollback procedure), is the runbook updated in the same PR, not as a follow-up that never happens?
Diagrams	Do sequence or architecture diagrams still reflect reality, or is this the change that makes the diagram lie? Diagrams-as-code (Mermaid, PlantUML) checked into the same repo as the change they describe age far better than static images maintained separately.
Decision logs / wiki	Is there a single source of truth being updated, or is this creating a second, soon-to-be-conflicting copy of information that already lives in the wiki?
Developer guides	If this PR changes a public interface or a common workflow, does the onboarding/developer guide for that area need a corresponding update?

The practical mechanism that makes this stick: a PR template checkbox ("Architecture Impact — has an ADR been created or updated?") costs nothing to add and, per practitioner guidance on ADR adoption, is one of the most effective low-friction ways to keep documentation review from being silently skipped.

Section 9 — ADR-Driven Development

9.1 What an ADR Is and Isn't

An Architecture Decision Record captures a single, architecturally significant decision — the context that made it necessary, the alternatives considered, and the consequences of the choice. The format was popularized by Michael Nygard in 2011 and has since been adopted from startups to national government digital services. The discipline is not the format (several templates compete: Nygard's original Context/Decision/Consequences, MADR, Y-Statements) — it is the habit of writing one down before the decision disappears into a PR nobody will reread.

Minimal Nygard-style template

```
# ADR-NNNN:

## Status
Proposed | Accepted | Superseded by ADR-XXXX

## Context
What is the problem or requirement forcing this decision?
What constraints (technical, business, team) apply?

## Decision
What are we doing, stated as a clear, single sentence if possible.

## Alternatives Considered
- Option A — pros / cons
- Option B — pros / cons

## Consequences
What becomes easier? What becomes harder? What follow-up
decisions does this create?
```

9.2 ADRs Before, During, and After Review

ADR Lifecycle Touchpoints

Phase	Practice
Before coding	An architecturally significant decision — new data store, new external dependency, a pattern that will be replicated elsewhere — gets an ADR in Proposed status before implementation starts, so the review of the idea happens before the review of hundreds of lines of code built on it.
During PR review	The reviewer's job shifts from re-litigating the decision to checking conformance: does the implementation match what the accepted ADR says? A reviewer who disagrees with the decision itself should reopen the ADR discussion, not block the PR with a design debate that was supposed to already be settled.

Phase	Practice
After merge	Some teams schedule a lightweight after-action review — commonly around one month later — comparing what the ADR predicted against what actually happened, closing the loop on whether the decision-making process itself is improving over time.

9.3 Traceability and Governance

- **Append-only, superseded not edited** — once accepted, an ADR is not rewritten; if the decision changes, a new ADR is written that explicitly supersedes the old one and links back to it, preserving the historical record of why direction shifted.
- **Central, discoverable storage** — ADRs kept in the same repository as the code they govern (commonly `docs/adr/``), in a lightweight diffable format (Markdown), so they show up in the same review and search tooling engineers already use.
- **One decision per record** — an ADR that documents "our entire database strategy" is a design document, not an ADR; each record should be short enough to read in the 10–15 minutes of a readout-style review meeting.
- **Confidence level** — recording how confident the team was at decision time (not just what was decided) gives future readers useful context for whether a decision is due for reconsideration.

9.4 Anti-Patterns

- ADRs written after the fact purely to satisfy a process checkbox, with alternatives listed that were never seriously considered — this produces documentation that looks rigorous but carries no real decision-making value.
- ADRs that never reach Accepted status and pile up as permanent Proposed clutter, which trains the team to stop reading them.
- Combining multiple unrelated decisions into a single ADR to save time, producing a document nobody can cleanly supersede later without also relitigating the unrelated parts.

Section 10 — RFC-Driven Engineering

10.1 RFCs vs. ADRs

Where an ADR records a decision already made, an RFC (Request for Comments) is the mechanism used to reach that decision for anything large enough to need broad, structured input before it's settled. Companies with strong public engineering cultures — including Cloudflare, Stripe, and large parts of Google and Meta's internal process — use RFC-style documents for anything that changes a widely-depended-on system, a public-facing product surface, or an internal platform other teams will build on.

10.2 How RFCs Affect PR Review

RFC Lifecycle and Its Relationship to PR Review

Aspect	Practice
Review lifecycle	An RFC typically moves through draft → circulated for comment → revised → approved by a named decision-maker or review board, with the comment period being the actual review — not a formality before an already-decided outcome.
Ownership	A named owner (often, but not always, the author) is responsible for driving the RFC to resolution and is the point of contact when a later PR appears to diverge from what was agreed.
Evolution	Once approved, an RFC — like an ADR — is not silently edited; material changes in direction get a follow-up RFC or an amendment section with a clear date stamp, preserving why the team believed what it believed at the time.
Downstream PR review	PRs implementing an approved RFC are reviewed for conformance to the RFC, with substantive design disagreement redirected back to an RFC amendment rather than re-litigated in PR comments — this is what keeps large, cross-team implementations from getting stuck in endless per-PR debate.

10.3 When to Use an RFC vs. an ADR vs. Neither

- **Use an RFC** when the decision affects multiple teams, is not yet settled, and needs broad input before it can be made — the RFC process itself is the decision-making mechanism.
- **Use an ADR** when a decision has already been made (by an individual, a small team, or as the outcome of an RFC) and needs to be recorded for future reference — the ADR process is a record-keeping mechanism, not a debate mechanism.
- **Use neither** for reversible, low-blast-radius decisions that a competent engineer or small team can make and adjust without ceremony — over-applying either process to routine decisions is the most common way organizations kill the practice's credibility.

About This Series

This is Volume 2 of a five-part Enterprise PR Review Playbook. Volume 1 covers workflow, ownership models, and role-based review playbooks. Volumes 3 and 4 cover AI-assisted and agentic review architectures. Volume 5 collects case studies, master checklists, and a review maturity model.

Generated as a synthesized practitioner reference. Standards references (OWASP Top 10:2025, SLSA) reflect the current published versions as of mid-2026; company-specific practices are drawn from public engineering blogs and evolve continuously.